



Taller de Lenguajes de Programación III — React Native

TRABAJO PRÁCTICO N.º 1

Docente a cargo: Rolón Lautaro Emanuel

Docente auxiliar: Mereles Juliana Aracelli

Ciclo lectivo: 2026

Fecha: 16/03/2026

Alumno: Ramírez Luana Abigail.

1.1 Node.js y NPM

1. ¿Qué es Node.js y en qué se diferencia de ejecutar JavaScript en el navegador?
2. Explicá con tus palabras la diferencia entre dependencies y devDependencies en el package.json. Dá un ejemplo de paquete para cada caso.
3. ¿Para qué sirve el archivo package-lock.json? ¿Por qué no se recomienda subir la carpeta node_modules a Git?
4. ¿Cuál es el punto de entrada principal en un proyecto? Ejemplo.
5. ¿Cómo se crea un proyecto Node.js con valores por defecto? Mostrá el comando utilizado en consola.
6. ¿Cómo se instala un paquete usando npm para que se agregue a las dependencies del proyecto? Mostrá un ejemplo.
7. ¿Qué es la carpeta node_modules y qué contiene dentro de un proyecto [Node.js](#)?
8. ¿Cuáles son los Scripts de NPM? Dar ejemplos.

Desarrollo:

1. [Node.js](#) es un entorno de ejecución para JavaScript, es una herramienta que te permite escribir y ejecutar código JS fuera del navegador web, directamente en una computadora o servidor. Algunas diferencias de ejecutarlo con JS en el navegador son que este opera en un entorno de cliente impredecible restringido para interactuar con interfaces gráficas, y [Node.js](#) se ejecuta del lado del servidor en un entorno controlado por el dev, permitiendo así el acceso directo al sistema operativo, archivos y bases de datos
2. Las **dependencies** son partes más esenciales del trabajo, sin ellas no funcionaría el proyecto y se borran alguna de ellas se rompería.

Ejemplos: express, postgre

Las **DevDependencies** son aquellas que facilitan el trabajo al desarrollador, estos son aquellas dependencies que se usan solo cuando se programa o desarrolla el trabajo

Ejemplos: Nodemon

3. El **package-lock.json** almacena un árbol de dependencies exacto y versionado, se genera automáticamente para cualquier operación en la que npm modifique el árbol node_modules o package.json. Describe el árbol exacto que se generó, de modo que las instalaciones posteriores puedan generar árboles idénticos, independientemente de las actualizaciones de dependencia intermedias. El **node_modules** no debe subirse, porque además de contener al menos 450 carpeta y pesar aprox. unos 75 MB, son bibliotecas de código externas administradas por

terceros, si los agregas no sólo estarás siguiendo tus cambios sino los cambios que tengan estos archivos en caso de alguna actualización de su propietario. Estos módulos y sus versiones específicas están ya descritos en el `package.json`, por eso es que cualquiera en posesión de ese archivo podrá reconstruir tu proyecto sin problemas.

4. Los puntos de entregas de los proyectos son el primer archivo que se ejecuta cuando se enciende el proyecto, convencionalmente se le llama “**main.js**” pero también se lo puede encontrar como “`index.js`”, “`app.js`” o “[servidor.js](#)”
5. Para crear un proyecto de [Node.js](#) con los valores por defecto, hay que abrir la consola o terminal dentro de la carpeta del proyecto y ejecutar el comando **npm init -y** (o **npm init -y**). El resultado es un `package.json` con el siguiente contenido:

```
{
  "name": "nombre",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

6. Para instalar un paquete y que se agreguen las dependencias del proyecto se usa el comando **npm install** o **npm i**, por ejemplo para instalar mongoose se haría “**npm install mongoose**”, en el `package.json` quedaría así:

```
{
  "name": "proyecto",
  "version": "1.0.0",
  "dependencies": {
    "mongoose": "^8.2.0"
  }
}
```

7. **node_modules** es la carpeta donde se almacenan todas las dependencias y librerías que utiliza el proyecto [node.js](#), suele ser una carpeta de tamaño grande y no es recomendada subirla a git, ya que npm tiene el comando `npm install`, el cual lee el

archivo package.json e instala las dependencias listadas en él para que el proyecto pueda funcionar.

8. Un script **npm** es una forma conveniente de agrupar comandos de shell comunes para un proyecto. Por lo general, son comandos o una cadena de comandos que normalmente se ingresarán en la línea de comandos para hacer algo con su aplicación. Los scripts se almacenan en el proyecto package.json archivo, lo que significa que se comparten entre todos los que usan el código base. Ayudan a automatizar tareas repetitivas y significan tener que aprender menos herramientas. Los scripts npm de Node también garantizan que todos usen el mismo comando con los mismos indicadores.

Ejemplo:

- `npm run server:`

```
"scripts": {  
  "server": "node server.js"  
}
```
- `npm run dev:`

```
"scripts": {  
  "dev": "node server.js"  
}
```

1.2 Bundlers: Vite y Webpack

1. ¿Qué son los Bundlers?
2. ¿Qué problema resuelve un bundler en el desarrollo de aplicaciones modernas?
3. ¿Cuál es el rol del Bundlers y optimización?
4. ¿Qué diferencias hay entre Vite y Webpack?
5. ¿Qué es Vite?

Desarrollo:

1. Un bundler “empaquetador” nos ayuda a empaquetar todo el código de nuestro proyecto (html, css, js), para que nosotros podamos subirlo a producción, es decir al dominio que tenemos en Internet.
- 2.

Fuentes:

<https://es.stackoverflow.com/questions/390486/qu%C3%A9-es-y-para-qu%C3%A9-me-sirve-la-carpeta-node-modules-en-mi-proyecto>

<http://es.stackoverflow.com/questions/309051/debo-o-no-incluir-la-carpeta-node-modules-de-ntro-de-las-carpetas-que-git-debe>

<https://stackoverflow.com/questions/44297803/what-is-the-role-of-the-package-lock-json>

<https://dev.to/ajitforger97/useful-npm-scripts-example-usage-2kha>

<https://heynode.com/tutorial/what-are-npm-scripts/>

<https://platzi.com/blog/bundlers-que-son-y-otras-opciones-hay-aybike/>